

master

...

problema-5---tipos-de-dados-Merus23 / problema-5-tipos.de.dados.md

 rcarocha feedback do professor 3 contributors   

200 lines (150 sloc) | 13 KB

...

Questão 1: Tipos Enumerados

Python não oferece **tipos enumerados** como abstração da linguagem mas oferece a classe `Enum` no módulo `enum`, que facilita a criação de enumerados quando o programador necessita. A descrição detalhada da classe `enum` encontra-se em <https://docs.python.org/3/library/enum.html>, onde também há um [exemplo](#).

Considere como exemplo adicional o código abaixo, também disponível no arquivo [exemplo-enum.py](#):

```
from enum import Enum

class DiasDaSemana(Enum):
    DOMINGO = 1
    SEGUNDA = 2
    TERCA = 3
    QUARTA = 4
    QUINTA = 5
    SEXTA = 6
    SABADO = 7

diaAula = [DiasDaSemana.SEGUNDA, DiasDaSemana.TERCA]

# calendario de aulas
for d in DiasDaSemana:
    if d in diaAula:
        print('AULA => ', d)
```

```
else:  
    print(' -- => ', d)
```

Esse código cria uma enumeração para descrever os dias da semana e na lista `diaAula` armazena os dias da semana com aula. O laço no fim do código exibe a lista dos dias da semana, indicando aqueles em que há aula, conforme a lista mencionada.

Considere as características de um tipo enumeração discutidas no capítulo 6, seção 6.4.1 ("[Tipos enumeração](#)", páginas 280 a 283) do livro "Conceitos de Linguagens de Programação".

Questões

1. Considerando as questões de projeto do uso de tipos enumerados (três questões na página 280), quais são as características ou escolhas da solução de Python?

Como mencionado no próprio enunciado, Python não fornece o tipo enumerado na abstração da linguagem. No entanto, a solução encontrada foi feita por meio de uma biblioteca chamada `enum`. Com isso, em Python, o tipo enumerado é declarado como uma classe com um parâmetro provido pela biblioteca 'enum', como por exemplo, `IntEnum`, `Enum`, `IntFlag` e etc. Desse modo, ainda durante o processo de declaração, é associado um número a uma palavra.

O acesso à classe pode ser feito de duas formas: índice e nome. O primeiro é passado como parâmetro para classe `DiasDaSemana(3)` e o segundo é acessando diretamente pelo nome `DiasDaSemana.SEGUNDA`. Considerando isso, as classes que são declaradas com o parâmetro `Enum`, são do tipo `enum` e seus atributos são membros da enumeração, não podendo ser convertido em alguns tipo, como por exemplo, `int` ou `float`. Entretanto, o mesmo não acontece ao tentar converter a classe `enum` para uma string, nesse caso, é mostrado o nome da classe e o nome do atributo ex.:

`DiasDaSemana.TERCA`. A seguir, os erros apresentados pelo terminal na tentativa de converter para o tipo `int` e `float` utilizando as funções `int()` e `float()`, respectivamente.

Tentativa de convertibilidade para o tipo `int` :

```
print(int(DiasDaSemana(1)))
```

```
Traceback (most recent call last):
```

```
File "main.py", line 14, in <module>
```

```
    print(int(DiasDaSemana(1)))
```

```
TypeError: int() argument must be a string, a bytes-like object or a  
number, not 'DiasDaSemana'
```

Tentativa de convertibilidade para o tipo `float` :

```
print(float(DiasDaSemana(1)))
```

Traceback (most recent call last):

File "main.py", line 14, in <module>

```
print(float(DiasDaSemana(1)))
```

TypeError: float() argument must be a string or a number, not 'DiasDaSemana'

Convertibilidade para o tipo `string` :

```
print(str(DiasDaSemana(1)))
```

```
DiasDaSemana.DOMINGO
```

Verificação com os tipos `int` , `float` e `string` utilizando `is` :

```
print(DiasDaSemana is int)
print(DiasDaSemana is float)
print(DiasDaSemana is str)
print('Com a passagem de parâmtros')
print(DiasDaSemana(1) is int)
print(DiasDaSemana(1) is float)
print(DiasDaSemana(1) is str)
```

```
False
```

```
False
```

```
False
```

```
Com passagem de parâmtros
```

```
False
```

```
False
```

```
False
```

Mesmo tendo sido mencionado acima, vale relembrar que, no caso do Python, a enumeração não é um tipo definido na abstração da linguagem, ou seja, não é possível transformar um tipo inteiro em um tipo enumerado e tão pouco transformar um tipo inteiro em uma classe enumerado.

2. Considerando a discussão quanto à avaliação da construção (seção 6.4.1.2, página 282 e 283), como a solução de Python pode ser avaliada em relação à legibilidade e confiabilidade?

A solução utilizada pelo Python, em relação à legibilidade, é muito parecida com a apresentada em outras linguagens como C e C#, isso do ponto de vista do programador que digita o código (desconsiderando questões relacionadas à abstração da linguagem). Ainda no que se refere à legibilidade, linguagens que fornecem esse tipo de abstração normalmente apresentam mais formas de declaração, em C e C#, por exemplo, a declaração do tipo enumerado pode ser feita de forma implícita e explícita. Em Python é utilizado apenas a segunda forma, o que não necessariamente implica em menos legibilidade. No entanto, é preciso destacar que a solução apontada em Python não é uma simples declaração de um tipo e sim a declaração de uma classe com o parâmetro `Enum`. Considerando um primeiro contato, é compreensível que o desenvolvedor não entenda o código na íntegra, embora provavelmente ele tenha uma forte ideia do que o código faz. Além disso, é válido salientar a legibilidade ao acessar uma classe enumerada em Python. Esse aspecto é bem interessante por causa do acesso por meio de índices, isto é, no exemplo dado, é possível acessar do domingo ao sábado por meio de um `for`, assim como mostrado abaixo:

```
for i in range(1,8):  
    print(DiasDaSemana(i))
```

Em relação à confiabilidade, a solução pensada no Python demonstra confiabilidade em alguns aspectos e em outros não. Primeiramente, a declaração da classe `enum` precisa ser feita de maneira explícita, o que possibilita ao programador informar, por exemplo, índices iguais sem que seja apresentada nenhuma mensagem de erro. Essa mesma representação pode ser encontrada em C ou C#, no entanto, nelas a declaração explícita é uma opção, ou seja, usualmente o programador não precisa se preocupar com isso. Abaixo um exemplo desse tipo de erro:

```
class DiasDaSemana(Enum):  
    DOMINGO = 1  
    SEGUNDA = 1  
    TERCA = 1  
    QUARTA = 1  
    QUINTA = 1  
    SEXTA = 1  
    SABADO = 1
```

Em segunda instância, mencionando agora aspectos positivos dessa solução, os membros enumerados são atômicos, isto é, não são divisíveis. Considerando isso, neste contexto, o atributo `DOMINGO` não é do tipo inteiro, o número `1` é utilizado apenas para apresentar a ordem desse atributo. Com isso, `DOMINGO` não pode ser operado aritmeticamente como um inteiro normalmente seria, tão pouco, convertido para outros tipos numerais (incluindo `int`). Ele pode ser convertido em uma string (`print(str(DiasDaSemana(1)))`) apresentando o nome da classe justamente com o atributo utilizado (`DiasDaSemana.DOMINGO`).

Declaração implícita em C:

```
#include <stdio.h>

int main()
{
    enum colors{red, blue, green};
}
```

Declaração explícita em C:

```
#include <stdio.h>

int main()
{
    enum colors{red = 1, blue = 2, green = 3};
}
```

Questão 2: Coleta de Lixo Automática em Python

Considere a implementação de uma lista duplamente ligada em Python, conforme indicado no código em (`lista_dl.py`). Se você não lembra ou não sabe o que é uma lista duplamente ligada, experimente [olhar neste artigo na Wikipedia](#).

A implementação foi mantida o mais simples e menos estruturada possível. Apenas as funções seguintes funções estão mostradas:

- `adiciona_fim_lista(lista, valor)` : adiciona o elemento com dado `valor` no fim da lista.
- `remove_iesimo(lista, ordem_iesimo)` : remove o `i`-ésimo elemento da lista, atualizando os ponteiros `proximo` e `anterior` na célula imediatamente anterior e posterior, respectivamente. O primeiro elemento é considerado de ordem 0.

- `remove_apartir_iesimo(lista, ordem_iesimo)` : remove todos os elementos da lista a partir do i-ésimo elemento. Se a lista possui 10 elementos, e é invocado `remove_apartir_iesimo` com a posição 5, então serão removidos os elementos 5, 6, 7, 8 e 9.

Questões

Considere as operações:

- `remove_iesimo(lista, ordem_iesimo)` .
- `remove_apartir_iesimo(lista, ordem_iesimo)`

1. Para cada operação (caso haja alguma diferença), explique como o coletor de lixo irá identificar os blocos `\[dado, proximo, anterior\]` que precisam ser desalocados e efetivamente desalocá-los da memória os algoritmos de coleta de lixo:

- i. Contagem de Referências
- ii. Algoritmo "Marcar e Varrer".

No caso apresentado, considerando as funções `remove_iesimo(lista, ordem_iesimo)` e `remove_apartir_iesimo(lista, ordem_iesimo)`, o primeiro parâmetro de cada função recebe uma variável na heap. Baseado nisso, o algoritmo contagem de referências verificará quantas referências as variáveis `dado`, `proximo` e `anterior` possuem.. Inicialmente, as três variáveis são inicializadas com `None`. Considerando as referências em cada uma das funções, a única variável na heap seria `proximo`, uma vez que ela recebe `lista.proximo = ListaDuplamenteLigada()`. Com isso, não seria necessário desalocar as outras variáveis, porque elas estão na pilha. Portanto, baseado na definição de cada uma dos algoritmos, o contador de referências verificaria a variável `proximo` e caso não houvesse nenhuma referência, a variável é desalocada. Porém, nesse contexto, a variável `proximo` terá uma referência durante toda a execução do código, isto é, ela não será desalocada utilizando o algoritmo de contagem de referências, isso acontece por causa de um problema já comentado em vídeo aula: referência circulares.

```
lista.proximo = ListaDuplamenteLigada()
```

O algoritmo `marcar e varrer`, por sua vez, tem uma abordagem mais direta. Ele marcará a variável `proximo` e caso não haja espaço na memória, a variável é desalocada.

2. Quais mudanças são necessárias nesses algoritmos para que o coletor de lixo identifique imediatamente a possibilidade de coleta de lixo, utilizando meramente o algoritmo com contagens de referências? Dê apenas uma visão geral das

mudanças que são necessárias. Não é necessário (e nem recomendado) modificar o código, mas você precisa fazer menção a partes do código na sua implementação.

Considerando a resposta anterior, haverá uma referência circular, ou seja, o algoritmo contador de referencias não será muito eficaz na coleta de lixo desse código. Para resolver essa questão, primeiro é preciso pensar em como esse algoritmo funciona, se o contador de referência de um objeto na heap está com o valor zero, então esse objeto é considerado lixo. Para solucionar isso, é necessário considerar duas observações: o objeto pode ser considerado lixo quando o contador dele for menor que zero; para os objetos considerados lixos, a contagem é feita por referências internas. Desse modo, objetos com valor de contagem abaixo de zero são candidatos a primeira observação. Com isso em mente, basta seguir três passos para resolver o problema: **marcar**, **varrer** e **coletar**. O primeiro marca apenas objetos com referências externas; o segundo varre a referência circular a partir de um objeto candidato com referência externa e restaura os outros objetos; por fim, os nós com contagem zero são coletados.

(apenas objetos com referências externas), **varrer** ()

Observação: /2.5Essa questão utiliza o Python como meramente uma ilustração, ao invés de considerar a maneira como a coleta de lixo no Python funciona e é implementada.

Feedback do Professor

Esta seção será escrita pelo professor ao final da avaliação da sua tarefa.

- Revisão prévia
 - **Q1.1:** Para elaborar a resposta de (2) (questão do livro) você deveria testar no código se essa conversão ocorre e aparentemente você não verificou. Faça um teste (uma linha de código) e mostre que essa conversão não é aceita. Ao contrário do que disse, o tipo `enum` não é uma abstração da linguagem: é provido por uma biblioteca.
 - **Q1.2:** Não sei o que você quer dizer com declaração implícita de enumerados em C (talvez seja C#?). O restante está ok.
 - **Q2.1.1-contagem:** nessa questão e na de "marcar-e-varrer", você respondeu aparentemente discutindo questões relacionadas ao código. A questão é relacionada ao algoritmo de coleta de lixo e, a questão pergunta o quanto que o código cria situações para dificultar a coleta de lixo. Pensei melhor na sua resposta.
 - **Q2.2:** Pense melhor na resposta considerando o comentário anterior.

- 'MATEUS PEREIRA DA SILVA' ==> 7,5 (problema Problema.5.tipos.de.dados)
- **Revisões/Problemas:**
 - A sua atividade foi submetida com **atraso de 5 dias**.
 - **Q1.1:** 2.5/2.5
 - **Q1.2:** 2.5/2.5
 - **Q2.1.1-contagem:** 1.25/1.25
 - **Q2.1.2-varrer:** 1.25/1.25
 - **Q2.2:** 0/2.5. Sem relação com a questão. Discuti com você em sala e pensei que não tivesse dificuldade em discutir.